Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 2

I41

QEMU

Machine View

```
void* checkFibonacci(void* arg) {
ThreadData* data = static_cast<ThreadData*>(arg);
int n = data->number;

if (isFibonacci(n))
std::cout << n << " is a Fibonacci member \n";
else
std::cout << n << " is not a Fibonacci member \n";
pthread_exit(nullptr);
}

int main(int argc, char* argv[]) {
if (argc < 2) {
std::cerr << "Usage: " << argv[0] << "  <positive integers...>\n";
return EXIT_FAILURE;
}
int numThreads = argc - 1;
pthread_t threads[numThreads];
ThreadData threadData[numThreads];

for (int i=0; i<numThreads; ++i) {
char* endptr;
long val = std::strtol(argv[i+1], &endptr, 10);
if (*endptr != '\0' || val <= 0) {
std::cerr << "Error: Invalid input " << argv[i+1] << " (must be a positive integer)\n";
return EXIT_FAILURE;
}
threadData[i].number = static_cast<int>(val);
int rc = pthread_create(&threads[i], nullptr, checkFibonacci, & threadData[i]);
if (rc) {
std::cerr << "Error: Unable to create thread (" << rc << ")\n";
return EXIT_FAILURE;
}
}
for (int i=0; i< numThreads; i++) {
pthread_join(threads[i], nullptr);
}
return EXIT_SUCCESS;
}



hitori@hitori:~$ g++ -O2 -Wall -pthread -o mtfibo main.cpp
hitori@hitori:~$ ./mtfibo 12 3 19 4
4 is not a Fibonacci member
19 is not a Fibonacci member
12 is not a Fibonacci member
3 is a Fibonacci member
```

I42

```
void* checkPrime(void* arg) {
ThreadData* data = static_cast<ThreadData*>(arg);
int n = data->number;
if (isPrime(n))
std::cout << n << " is a Prime \n";
else
std::cout << n << " is not a Prime \n";
pthread_exit(nullptr);
}

int main(int argc, char* argv[]) {
if (argc < 2) {
std::cerr << "Usage: " << argv[0] << "  <positive integers...>\n";
return EXIT_FAILURE;
}
int numThreads = argc - 1;
pthread_t threads[numThreads];
ThreadData threadData[numThreads];

for (int i=0; i<numThreads; ++i) {
char* endptr;
long val = std::strtol(argv[i+1], &endptr, 10);
if (*endptr != '\0' || val <= 0) {
std::cerr << "Error: Invalid input " << argv[i+1] << " (must be a positive integer)\n";
return EXIT_FAILURE;
}
threadData[i].number = static_cast<int>(val);
int rc = pthread_create(&threads[i], nullptr, checkPrime, & threadData[i]);
if (rc) {
std::cerr << "Error: Unable to create thread (" << rc << ")\n";
return EXIT_FAILURE;
}
}
for (int i=0; i< numThreads; i++) {
pthread_join(threads[i], nullptr);
}
return EXIT_SUCCESS;
}



hitori@hitori:~$ g++ -O2 -Wall -pthread -o mtprime main.cpp
hitori@hitori:~$ ./mtprime 12 3 19 4
4 is not a Prime
19 is a Prime
12 is not a Prime
3 is a Prime
```

I43

```
struct ThreadData {
int number;
};

bool isGrowingNumber(int n) {
if (n < 10) return false;

std::vector<int> digits;
while (n > 0) {
digits.push_back(n % 10);
n /= 10;
}
for (int i = digits.size() - 1; i > 0; --i) {
if (digits[i] >= digits[i - 1]) {
continue;
} else {
return false;
}
}
return true;
}

void* checkGrowing(void* arg) {
ThreadData* data = static_cast<ThreadData*>(arg);
int n = data->number;
if (isGrowingNumber(n))
std::cout << n << " is a growing number \n";
else
std::cout << n << " is not a growing number \n";
pthread_exit(nullptr);
}

int main(int argc, char* argv[]) {
if (argc < 2) {
std::cerr << "Usage: " << argv[0] << "  <positive integers...>\n";
return EXIT_FAILURE;
}
int numThreads = argc - 1;
pthread_t threads[numThreads];
ThreadData threadData[numThreads];



hitori@hitori:~$ g++ -O2 -Wall -pthread -o grow main.cpp
hitori@hitori:~$ ./grow 1 23 144 3689
3689 is not a growing number
144 is not a growing number
1 is not a growing number
23 is not a growing number
```

l31

```
int main(int argc, char* argv[]) {
if (argc < 2) {
std::cerr << "Usage: " << argv[0] << "  <positive integers...>\n";
return EXIT_FAILURE;
}
int numNumbers = argc - 1;
int status;

for (int i=0; i<numNumbers; ++i) {
char* endptr;
long val = std::strtol(argv[i+1], &endptr, 10);
if (*endptr != '\0' || val <= 0) {
std::cerr << "Error: Invalid input " << argv[i+1] << " (must be a positive integer)\n";
return EXIT_FAILURE;
}

pid_t pid = fork();

if (pid < 0) {
perror("fork failed");
return EXIT_FAILURE;
} else if (pid == 0) {
int n = static_cast<int>(val);
if (isFibonacci(n))
std::cout << n << " is a Fibonacci member\n";
else
std::cout << n << " is not a Fibonacci member\n";

exit(EXIT_SUCCESS);
}
}
while (wait(&status) > 0);
return EXIT_SUCCESS;
}
```

```
hitori@hitori:~$ g++ -O2 -Wall -lm -o mpfibo main.cpp
hitori@hitori:~$ ./mpfibo 12 3 19 4
12 is not a Fibonacci member
19 is not a Fibonacci member
3 is a Fibonacci member
4 is not a Fibonacci member
```

I32

```
  GNU nano 6.2
#include <iostream>
#include <cstdlib>
#include <cmath>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

bool isFibonacci(int n) {
if (n < 0) return false;
long long x1 = 5LL * n * n + 4;
long long x2 = 5LL * n * n - 4;
long long s1 = std::sqrt(x1);
long long s2 = std::sqrt(x2);
return (s1 * s1 == x1) || (s2 * s2 == x2);
}
```

```
  GNU nano 6.2
#include <iostream>
#include <cstdlib>
#include <cmath>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

bool isPrime(int n) {
if (n <= 1) return false;
if (n == 2) return true;
if (n % 2 == 0) return false;

int limit = static_cast<int>(std::sqrt(n));
for (int i = 3; i <= limit; i += 2) {
if (n % i == 0) return false;
}
return true;
}
```

```
int main(int argc, char* argv[]) {
if (argc < 2) {
std::cerr << "Usage: " << argv[0] << "  <positive integers...>\n";
return EXIT_FAILURE;
}
int numNumbers = argc - 1;
int status;

for (int i=0; i<numNumbers; ++i) {
char* endptr;
long val = std::strtol(argv[i+1], &endptr, 10);
if (*endptr != '\0' || val <= 0) {
std::cerr << "Error: Invalid input " << argv[i+1] << " (must be a positive integer)\n";
return EXIT_FAILURE;
}

pid_t pid = fork();

if (pid < 0) {
perror("fork failed");
return EXIT_FAILURE;
} else if (pid == 0) {
int n = static_cast<int>(val);
if (isPrime(n))
std::cout << n << " is a Prime\n";
else
std::cout << n << " is not a Prime\n";


hitori@hitori:~$ g++ -O2 -Wall -lm -o mpprime main.cpp
hitori@hitori:~$ ./mpprime 12 3 19 4
3 is a Prime
12 is not a Prime
19 is a Prime
4 is not a Prime
```

l33

```
#include <sys/types.h>
#include <sys/wait.h>

bool isGrowingNumber(int n) {
if (n < 10) return false;

std::vector<int> digits;
while (n > 0) {
digits.push_back(n % 10);
n /= 10;
}
for (int i = digits.size() - 1; i > 0; --i) {
if (digits[i] >= digits[i - 1]) {
continue;
} else {
return false;
}
}
return true;
}

int main(int argc, char* argv[]) {
if (argc < 2) {
std::cerr << "Usage: " << argv[0] << "  <positive integers...>\n";
return EXIT_FAILURE;
}
int numNumbers = argc - 1;
std::vector<pid_t> pids;

for (int i=0; i<numNumbers; ++i) {
char* endptr;
long val = std::strtol(argv[i+1], &endptr, 10);
if (*endptr != '\0' || val <= 0) {
std::cerr << "Error: Invalid input " << argv[i+1] << " (must be a positive integer)\n";
return EXIT_FAILURE;
}

pid_t pid = fork();

if (pid < 0) {
perror("fork failed");
return EXIT_FAILURE;


hitori@hitori:~$ ./grow 1 23 144 3689
1 is not a growing number on process1668
3689 is not a growing number on process1671
144 is not a growing number on process1670
23 is not a growing number on process1669
```

```
if (pid < 0) {
perror("fork failed");
return EXIT_FAILURE;
} else if (pid == 0) {
int n = static_cast<int>(val);
if (isGrowingNumber(n))
std::cout << n << " is a growing number on process" <<getpid() << "\n";
else
std::cout << n << " is not a growing number on process" <<getpid() << "\n";

exit(EXIT_SUCCESS);
} else {
pids.push_back(pid);
}
}
for (pid_t pid : pids) {
int status;
waitpid(pid, &status, 0);
}
return EXIT_SUCCESS;
}
```